

SQL Database Generation Project Report

Synthetic E-Commerce Database using SQLite Name:

Ajayi Israel

1. INTRODUCTION / PURPOSE OF THE DATABASE

This project involves designing and creating a synthetic relational database using SQLite. The goal of the store database is to simulate a e-commerce site where customers can browse for items, purchase them and then leave reviews for the items they buy. The goal of the assignment is to demonstrate knowledge of SQL database design, data generation techniques, and relational database structures.

Database for database assignment This database is a simplified relational database for an online retail store. It has tables for storing customers, products, customer orders, individual order items and product reviews. The **customers**, **products**, **orders**, **order_items** and **reviews** tables are the five main tables in this database. The purpose of these tables is to represent the relationship between customers and the products they purchase. Using this database we can implement real-life business scenarios such as order tracking, product sales analysis and customer review analysis.

2. DATA GENERATION METHOD

The data for this assignment was produced programmatically using Python in a Google Colab notebook. Rather than downloading an existing dataset the data was produced using Python and custom logic. The dataset was produced from scratch to enable full control over the data. The data was also produced specifically for this assignment.

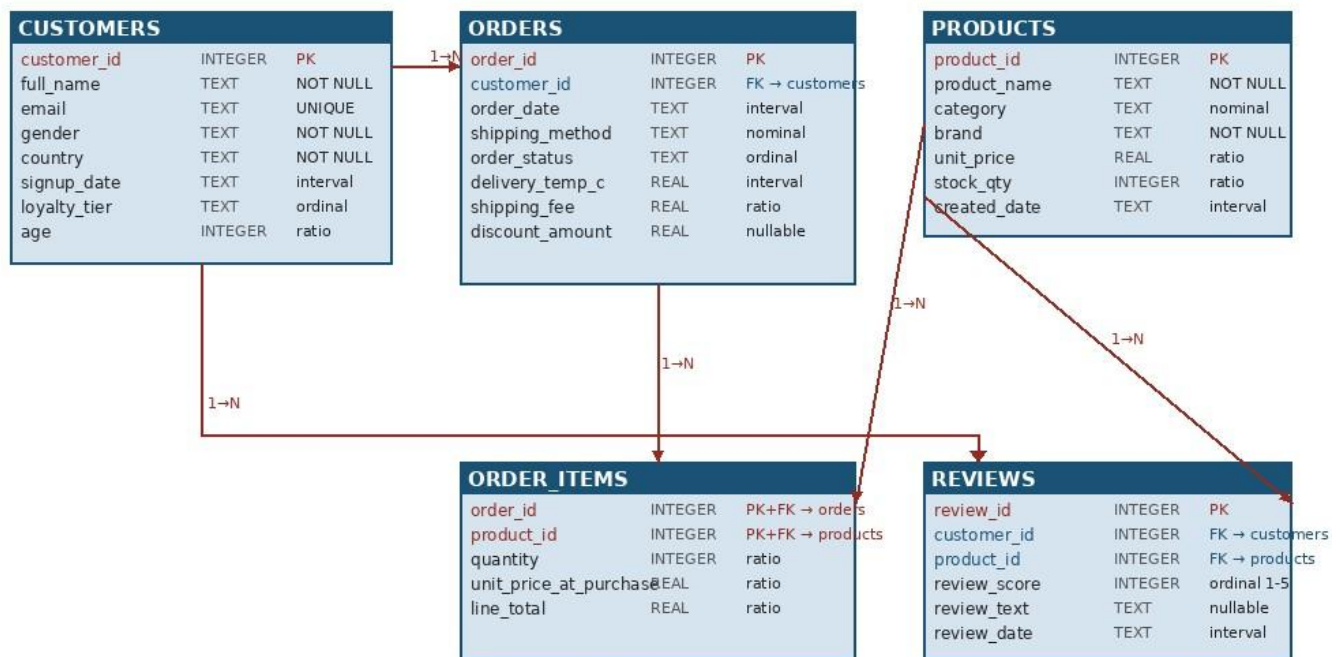
A number of Python libraries were employed when creating our data. The Faker library was used to create names and email addresses of customers, all through the UK locale so that all the names and email domains are suitably representative of a UK customer base. Random numbers from the Python random library were also used to generate the numbers for the ages of customers, product prices, shipping costs and quantities in an order. Weighted probability distributions were applied to produce realistic data patterns — for example, most orders were assigned the status “Delivered”, reflecting real-world e-commerce behaviour.

To further enhance realism, the dataset deliberately includes **missing values** and **duplicate values**. Approximately 5% of customer email addresses were intentionally set to NULL to simulate incomplete records. Around 20% of orders have NULL discount values to represent orders where no promotional discount was applied. Approximately 40% of product reviews contain no review text, reflecting realworld patterns where customers submit star ratings without written feedback. Duplicate values were also introduced in customer names to simulate different individuals sharing the same name.

Realistic statistical distributions were also applied. Customer ages were generated using a normal distribution centred around typical adult ages and clamped to a valid range (16–90). Product prices followed a skewed distribution so that most products fall within common price ranges while a small number of higher-priced items exist, particularly in the Electronics category.

3. DATABASE SCHEMA

The database schema consists of five relational tables that represent the core components of an ecommerce system. These tables are connected through foreign key relationships to maintain referential integrity between entities. The entity relationship diagram below illustrates the structure and relationships between tables.



Entity Relationship Diagram — Synthetic E-Commerce SQLite Database

■ PK = Primary Key ■ FK = Foreign Key → arrows show foreign key direction

Figure 1: Entity Relationship Diagram — Synthetic E-Commerce SQLite Database **customers:** Stores customer information. Primary key: **customer_id**. Contains **full_name**, **email** (UNIQUE, nullable), **gender**, **country**, **signup_date**, **loyalty_tier**, and **age**.

products: Stores the product catalogue. Primary key: **product_id**. Contains **product_name**, **category**, **brand**, **unit_price**, **stock_qty**, and **created_date**.

orders: Records each customer purchase. Primary key: **order_id**. Foreign key: **customer_id** links to **customers**. Stores **order_date**, **shipping_method**, **order_status**, **delivery_temp_c**, **shipping_fee**, and **discount_amount**.

order_items: Bridge table resolving the many-to-many relationship between **orders** and **products**. Uses a composite primary key (**order_id**, **product_id**). Stores **quantity**, unit price at time of purchase, and **line_total**.

reviews: Stores customer product feedback. Primary key: **review_id**. Foreign keys reference both **customers** and **products**. Stores **review_score** (ordinal, 1–5), optional **review_text**, and **review_date**.

4. CONSTRAINTS AND DATA TYPES

Several constraints were implemented in the database schema to ensure data integrity. Primary keys uniquely identify each record across all tables. Foreign key constraints enforce referential integrity — for example, the **customer_id** in **orders** references the **customers** table, and **order_items** table references both **orders** and **products**. NOT NULL constraints ensure essential attributes are always present. A UNIQUE constraint on **email** prevents duplicate addresses when provided. CHECK constraints restrict values to valid ranges: **age** is bounded between 16 and 90, **review_score** between 1 and 5, and prices and quantities are restricted to non-negative values.

The database covers all four measurement scales. Nominal data appears in attributes such as **gender**, **country**, **category**, **brand**, and **shipping_method**. Ordinal data appears in **loyalty_tier** (Bronze, Silver, Gold, Platinum) and **review_score** (1–5). Ratio data includes **age**, **unit_price**, **stock_qty**, **quantity**, **shipping_fee**, and **line_total**, where zero has a meaningful interpretation. Interval data is represented by date fields (**signup_date**, **order_date**, **review_date**) and **delivery_temp_c**, where differences between values are meaningful but zero does not represent an absolute absence.

5. ETHICAL AND DATA PRIVACY CONSIDERATIONS

Ethics was a major concern in the database design. We created synthetic data that is not real or relative to any person or business. This was done to make sure that there would be no privacy problems regarding personal information.

No real customer names, emails or other personal identifiable information was collected or stored. Instead we used Faker to generate fictional data that seems real. This way our dataset is GDPR compliant. The General Data Protection Regulation (GDPR) lays out the rules for how to collect, store and use personal data, for example that the personal data must be necessary and proportionate to the purpose it is being used for. In this case using fake data means we don't have to worry about any of those rules, and we still end up with a very similar structure to real data.

As we have a real database at our disposal, we should think about the risks in case of a real database with real customer data. In this case, the personal data according to GDPR would be name, email, purchase history and geographic location. A data breach could cause serious damage to customers, such as identity theft, fraud or unauthorised profiling. In case of an e-commerce database, companies would have to implement appropriate measures in order to secure their data, like authentication/authorisation, encryption and data deletion policies.

This section implements the GDPR right to erasure (right to be forgotten). Let us say a customer wishes us to delete his personal information. The database must be able to erase his customer data, and in doing so erase any associated orders and reviews. This is achieved by defining the GDPR right to erasure by properly implementing ON DELETE CASCADE on the order_items foreign key.

The Data Minimisation principle that we referred to above. An "actual" ecommerce system would in practice collect only the information required for it to carry out its specified purpose. Hence storing a customer's real date of birth as opposed to simply a age range and/or collecting all of a credit/debit card's digits as opposed to just the first 6 digits for an online payment would all increase an ecommerce system's overall privacy vulnerability. These choices have been therefore be avoided when populating our fictional "database".

The use of synthetic data in the classroom is very important. Using synthetic data allows students to play and experiment with the schema of databases and data analysis techniques without dealing with the ethical and legal issues arising from the use of personal real data